— PREPRINT NOT SUBMITTED. PLEASE DO NOT REDISTRIBUTE —

EFFICIENT STRUCTURED MULTIFRONTAL FACTORIZATION FOR LARGE SPARSE MATRICES

JIANLIN XIA*

Abstract. This paper presents structured multifrontal factorization algorithms for large sparse matrices. The algorithms use the idea that during the direct factorization of some discretized matrices, certain dense intermediate matrices have a low-rank property and thus can be approximated by compact structures. We take full advantage of sparse matrix techniques by organizing the factorization with a supernodal version multifrontal method and the nested dissection ordering of the matrix. Each dense intermediate matrix is formed explicitly and then partially factorized with hierarchically semiseparable (HSS) matrix operations. Some new improved HSS algorithms are proposed. Simple and fast structured operations for conducting the multifrontal method are developed, including new partial HSS factorization, Schur complement computation via low-rank updates, and structured solution. We also relax the classical rank requirement for HSS matrix algorithms so as to achieve the same or similar performance with more flexible structures. The overall structured multifrontal factorization is much simpler than similar structured methods, and is more generally applicable. Both a robust version and a scalable version are given. The robust version guarantees the existence of the factorization regardless of the accuracy of the structured approximation. The methods have nearly linear ($O(N)$) complexity for some 2D discretized matrices, and roughly $O(N^{4/3})$ complexity for some 3D ones. The methods also have the potential to work as effective preconditioners even if the low-rank property is not significant. We demonstrate the reliability and effectiveness of the methods with various applications.

Key words. supernodal multifrontal method, robust approximate factorization, Schur compensation, HSS structure, low-rank property

AMS subject classifications. 15A23, 65F05, 65F30, 65F50

1. Introduction.

Large sparse linear systems often arise from numerical solutions of PDEs in practical problems. Typically, there are two types of linear system solvers, direct methods and iterative methods. Direct methods are reliable and are efficient for multiple right-hand sides, but are often expensive due to the generation of fill-in or loss of sparsity. Iterative methods take good advantage of sparsity and require less storage, but may diverge or converge slowly without effective preconditioners.

In this paper, we are interested in fast approximate factorizations of large discretized matrices using structured operations. These factorizations can be used as direct solvers with specified accuracy, or as preconditioners. Assume we have a system

$$A\mathbf{x} = \mathbf{b} \tag{1.1}$$

arising from the discretization of certain PDEs, where A is symmetric positive definite (SPD). In the direct solution of the system, the factorization of A generally create new nonzeros or fill-in in the factor. One way to reduce fill-in is to reorder the rows and columns of A, or to reorder the mesh points used in the discretization. For example, for an $n \times n$ two-dimensional (2D) regular mesh, the nested dissection ordering [26] and its generalizations lead to $O(N^{3/2})$ complexity factorizations with $O(N \log N)$ storage, where $N = n^2$ is the order of A. For 2D problems, these are shown to be lower bounds for exact factorizations with any ordering [34] (ignoring special techniques such as Strassen's algorithm [46]). On the other hand, some iterative methods such as multigrid converges with $O(N)$ complexity for some problems.

*Department of Mathematics, Purdue University, West Lafayette, IN 47907 (xiaj@math.purdue.edu).

In the recent years, nearly linear complexity structured approximate factorization methods have been developed based on a low-rank property. It has been noticed [3, 4, 9, 28, 29, 49, etc.] that for some PDEs such as elliptic equations, the fill-in in the direct factorization of A has a low-rank property. That is, certain off-diagonal blocks have small numerical ranks. This property can be used to improve the efficiency of the factorization. Rank structured matrices such as quasiseparable, semiseparable, or hierarchical matrices [25, 48, 30] can be used to approximate the dense intermediate matrices.

[49] proposes a structured sparse factorization algorithm based on the multifrontal method [24, 37] and hierarchically semiseparable (HSS) matrix representations [10, 11, 16]. The algorithm in [49] has nearly linear complexity and linear storage requirement. It uses structured methods for most major operations. Many complicated procedures (structure generation, dense structured factorization, permutation, splitting, merging, compression, etc.) are involved, and this makes the algorithm difficult to implement. The current implementation can only work on 2D regular grids. Also, its may suffer from the problem of breakdown, or the lose of positive definiteness, especially when a large tolerance is used. In addition, the structured operations have poor scalability.

This paper proposes a much simpler and more general algorithm. It includes new improved dense structured methods, relaxed rank requirement, and flexible and simple factorization framework.

First, for dense intermediate fill-in, we give some new improved HSS operations, including the structure generation and dense factorization. Both the efficiency and the scalability are considered. We also provide an option of using a robust HSS triangular factorization in [50] to handle the dense fill-in.

Second, we systematically relax the classical rank requirement of general HSS operations. Traditionally, HSS operations often require the related off-diagonal (numerical) ranks to be bounded in order to achieve certain complexity. Here, we show that we can obtained the same or similar complexity without this requirement. Or, the ranks are actually allowed to increase along the block sizes. This brings more flexibility and broader opportunities to using HSS matrices.

Finally, the overall sparse factorization then uses the multifrontal method together with the new HSS algorithms or the robust HSS triangular factorization in [50]. We organize the overall factorization with a supernodal version multifrontal method together with the nest dissection ordering of mesh points. The multifrontal method is a very important direct methods for sparse matrix solutions [24, 37]. It reorganizes the sparse factorizations as local factorizations of small dense matrices in terms of frontal and update matrices. It thus has nice data locality and takes good advantage of dense block operations. Furthermore, it enables us to conveniently use HSS matrices to approximate the dense blocks. Unlike the method in [49], here, the dense intermediate frontal matrices are first formed as dense blocks and are then approximately factorized into structured forms. Schur complements (update matrices) are quickly computed via low-rank updates. The assembly of update matrices (called extend-add) remains non-structured. Therefore, the only major HSS operations are structure generation and factorization, and the entire sparse factorization is significantly simpler than the one in [49]. It is also more general and does not require the mesh to be regular. In addition, we can more conveniently integrate parallelizable operations and/or robust techniques. New formulas are derived for quickly computing the update matrices after the partial factorization of frontal matrices.

With certain rank condition (much looser than the one in [49]), the total fac-

torization complexity of this new algorithm is similar to the one in [49]. But the new algorithm applies to more general sparse problems including 3D problems. The cost for 2D problems is roughly linear in N (see Theorem 4.3), and for 3D problems, roughly $O(N^{4/3})$ flops (see Theorem 4.4). The storage requirement is nearly $O(N)$. Sample applications include (but are not limited to) Poisson's equations in both 2D and 3D, according to the rank analysis in [9]. In contrast, the exact direct factorization generally costs at least $O(N^{4/3})$ in 2D and $O(N^2)$ in 3D.

The cost of solving systems with the structured factors is nearly $O(N)$. Such structured factors are represented by two layers of tree structures, an outer assembly tree for the multifrontal method, and an inner HSS tree corresponding to each frontal matrix. This type of structured solvers are very attractive for real applications such as Helmholtz equations in seismic imaging, where linear systems are solved with a large number of right-hand sides but only modest accuracy is desired.

The remaining sections are organized as follows. Section 2 focuses on dense structured factorization and proposes some new HSS algorithms. Relaxed rank requirements for HSS operations are discussed in Section 3. New structured multifrontal factorization and solution algorithms are developed in Sections 4 and 5, respectively. Section 6 provides some numerical examples. We draw some concluding remarks in Section 7.

2. Dense structured factorization.

2.1. Review of hierarchically semiseparable structures. HSS structures are useful in handling dense matrices with the low-rank property. The HSS representation is generally defined recursively [10, 11, 16]. The definition for a postordering HSS form [16, 50] is as follows.

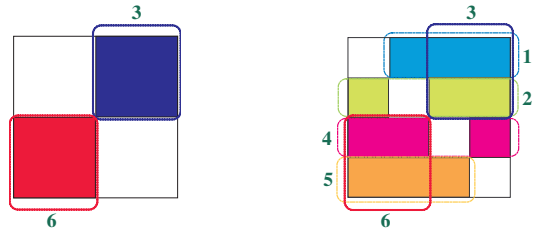
Assume F be an $N \times N$ dense matrix, and $\mathcal{I} = \{1, 2, \ldots, N\}$. Let T be a binary tree with k nodes, and $t_i \subset T$ be an index set associated with each node i of T. We say T is a *postordered HSS tree* if it satisfies:

1. T is a full binary tree: each node i is either a leaf or is a non-leaf node with two children $c_{i;1}$ and $c_{i;2}$ (or c_1 and c_2 for simplicity).
2. T is in its postordering: children are ordered before parents, and a left node is ordered before its sibling.
3. For each non-leaf node i, $t_{c_{i;1}} \cup t_{c_{i;2}} = t_i$, $t_{c_{i;1}} \cap t_{c_{i;2}} = \phi$, and $t_k = \mathcal{I}$.
4. There exists matrices $D_i, U_i, V_i, R_i, W_i, B_i$ (called HSS generators) associated with each node i satisfying

$$D_i = \begin{pmatrix} D_{c_{i;1}} & U_{c_{i;1}} B_{c_{i;1}} V_{c_{i;2}}^T \\ U_{c_{i;2}} B_{c_{i;2}} V_{c_{i;1}}^T & D_{c_{i;2}} \end{pmatrix}, \; U_i = \begin{pmatrix} U_{c_{i;1}} R_{c_{i;1}} \\ U_{c_{i;2}} R_{c_{i;2}} \end{pmatrix}, \; V_i = \begin{pmatrix} V_{c_{i;1}} W_{c_{i;1}} \\ V_{c_{i;2}} W_{c_{i;2}} \end{pmatrix},$$

so that $D_i \equiv F|_{t_i \times t_i}$. Here, $D_k \equiv F$, and R_i, W_i, and B_i are empty matrices.

Then, F is in an HSS form given be the generators. For a non-leaf node i, the generators D_i, U_i, V_i are recursively defined and are not explicitly stored. Clearly, U_i is a column basis for the off-diagonal block row $F|_{t_i \times (\mathcal{I} \setminus t_i)}$, and V_i is a row basis for the off-diagonal block column $F|_{(\mathcal{I} \setminus t_i) \times t_i}$. These off-diagonal blocks are called *HSS blocks* (Figure 2.1). (For simplicity, later, we often just consider HSS block rows.) U_i and V_i are also called cluster bases in [7, 8].



(i) One level of HSS blocks (ii) Two levels of HSS blocks

FIG. 2.1. *Two levels of HSS (off-diagonal) blocks.*

The following is a block 4×4 HSS matrix example:

(2.1)
$$F = \begin{pmatrix} D_1 & U_1 B_1 V_2^T & U_1 R_1 B_3 W_4^T V_4^T & U_1 R_1 B_3 W_5^T V_5^T \\ U_2 B_2 V_1^T & D_2 & U_2 R_2 B_3 W_4^T V_4^T & U_2 R_2 B_3 W_5^T V_5^T \\ U_4 R_4 B_6 W_1^T V_1^T & U_4 R_4 B_6 W_2^T V_2^T & D_4 & U_4 B_4 V_5^T \\ U_5 R_5 B_6 W_1^T V_1^T & U_5 R_5 B_6 W_2^T V_2^T & U_5 B_5 V_4^T & D_5 \end{pmatrix}.$$

See Figure 2.2. If F is symmetric, we can set [16]

$$D_i = D_i^T, \ V_i = U_i, \ B_j = B_i^T, \tag{2.2}$$

where j is the sibling of i.

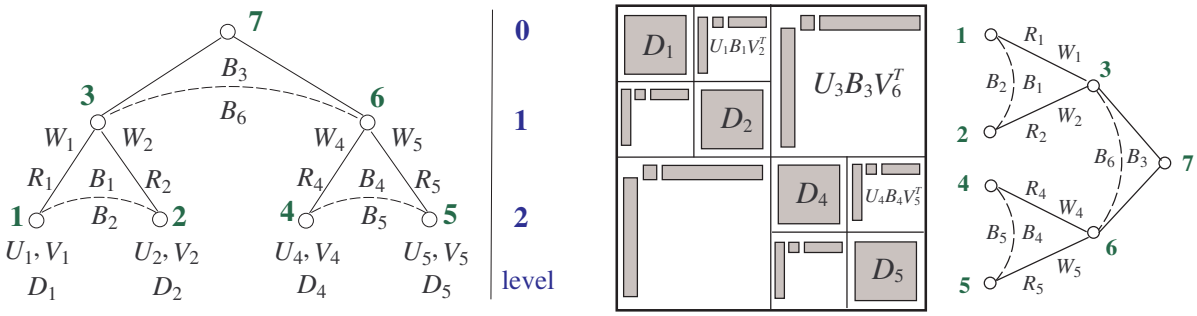


FIG. 2.2. *HSS tree for (2.1) and the multi-level matrix pattern.*

We then discuss two types of HSS factorizations, a robust structured Cholesky factorization with the factor in HSS form, and a parallel ULV type factorization or a generalized HSS Cholesky factorization.

2.2. Robust HSS Cholesky factorization. The factorization method in [50] computes an approximate Cholesky factorization for an $N \times N$ dense SPD matrix $F \approx \mathbf{L}\mathbf{L}^T$ where $\mathbf{L}$ is a lower triangular HSS matrix. The approximation $\mathbf{L}\mathbf{L}^T$ is guaranteed to exist and to be positive definite, regardless of the accuracy. The fundamental idea can be illustrated by a block 2×2 example

$$F \equiv \begin{pmatrix} F_{11} & F_{21}^T \\ F_{21} & F_{22} \end{pmatrix}. \tag{2.3}$$

Compute a Cholesky factorization $F_{11} = D_1 D_1^T$ and an SVD

$$D_1^{-1} F_{21}^T = \begin{pmatrix} U_1 & \hat{U}_1 \end{pmatrix} \begin{pmatrix} B_1 & \\ & \hat{B}_1 \end{pmatrix} \begin{pmatrix} U_2^T \\ \hat{U}_2^T \end{pmatrix} = U_1 B_1 U_2^T + \hat{U}_1 \hat{B}_1 \hat{U}_2^T,$$

where $B_1 = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$, $\hat{B}_1 = \mathrm{diag}(\sigma_{r+1}, \ldots, \sigma_N)$ and $\sigma_1 \geq \cdots \geq \sigma_r \geq \tau \geq \sigma_{r+1} \geq \cdots \geq \sigma_m$. Then

$$F = \begin{pmatrix} D_1 & \\ U_2 B_1^T U_1^T + \hat{U}_2 \hat{B}_1^T \hat{U}_1^T & I \end{pmatrix} \begin{pmatrix} D_1^T & U_1 B_1 U_2^T + \hat{U}_1 \hat{B}_1 \hat{U}_2^T \\ & S \end{pmatrix}.$$

where $S = F_{22} - U_2^T B_1^T B_1 U_2 - \hat{U}_2 \hat{B}_1^T \hat{B}_1 \hat{U}_2^T$ is the Schur complement. If all the singular values in $\hat{B}_1$ are dropped (so that the numerical rank of F_{21}^T is r), S is approximated by

$$\tilde{S} = F_{22} - U_2^T B_1^T B_1 U_2 = S + \hat{U}_2 \hat{B}_1^T \hat{B}_1 \hat{U}_2^T = S + O(\tau^2). \tag{2.4}$$

That is, a positive semidefinite term is automatically added to the Schur complement. the Schur complement. Then compute a Cholesky factorization $\tilde{S} = D_2 D_2^T$ and we have

$$F \approx \mathbf{L}\mathbf{L}^T = \begin{pmatrix} D_1 & \\ U_2 B_2 U_1^T & D_2 \end{pmatrix} \begin{pmatrix} D_1^T & U_1 B_1 U_2^T \\ & D_2^T \end{pmatrix}.$$

This idea of Schur compensation can be generalized to multiple blocks so as to compute an approximate HSS factorization. The factorization is conducted along the postordering traversal of a given HSS tree. The HSS blocks of L are computed and then compressed at multiple levels. See [50] for the details. The algorithm gives an HSS form $\mathbf{L}$ where all the U generators have orthonormal columns.

2.3. Scalable generalized HSS Cholesky factorization. On the other hand, we can also first construct an HSS approximation to F and then use the generalized HSS Cholesky factorization [16]. These two steps are both parallelizable.

2.3.1. Scalable symmetric HSS construction. The HSS construction uses nested compression similar to the one in [16]. However, the construction algorithm in [16] is a sequential process since early compression results participate in all the later compression. Here, we propose a new algorithm with better scalability. There are two stages.

During the first stage, all HSS block rows are compressed. If i is a leaf, compute an RRQR

$$F|_{t_i \times (\mathcal{I} \setminus t_i)} \approx U_i \tilde{F}|_{\tilde{t}_i \times (\mathcal{I} \setminus t_i)}.$$

If i is a non-leaf node with children c_1 and c_2, compute an RRQR of the matrix

$$\begin{pmatrix} \tilde{F}|_{\tilde{t}_{c_2} \times (\mathcal{I} \setminus t_i)} \\ \tilde{F}|_{\tilde{t}_{c_2} \times (\mathcal{I} \setminus t_i)} \end{pmatrix} \approx \begin{pmatrix} R_{c_1} \\ R_{c_2} \end{pmatrix} \tilde{F}|_{\tilde{t}_i \times (\mathcal{I} \setminus t_i)}.$$

Then clearly,

$$F|_{t_i \times (\mathcal{I} \setminus t_i)} \approx \begin{pmatrix} U_{c_1} \tilde{F}|_{\hat{t}_{c_2} \times (\mathcal{I} \setminus t_i)} \\ U_{c_2} \tilde{F}|_{\hat{t}_{c_2} \times (\mathcal{I} \setminus t_i)} \end{pmatrix} = \begin{pmatrix} U_{c_1} & \\ & U_{c_2} \end{pmatrix} \begin{pmatrix} R_{c_1} \\ R_{c_2} \end{pmatrix} \tilde{F}|_{\tilde{t}_i \times (\mathcal{I} \setminus t_i)} = U_i \tilde{F}|_{\tilde{t}_i \times (\mathcal{I} \setminus t_i)}.$$

That is, we obtain the orthonormal column basis U_i for each HSS block row $F|_{t_i \times (\mathcal{I} \setminus t_i)}$. These compression steps are done independently of other nodes at the same level of the HSS tree. After this stage, all U and R generators are available.

In the next stage, we compute the B generators. Let i be a right node and j be the sibling of i. Then according to the first stage,

$$F|_{t_j \times t_i} \approx U_j \tilde{F}|_{\tilde{t}_j \times t_i}.$$

On the other hand, in the HSS approximation, $F|_{t_j \times t_i}$ has a form $U_j B_i^T U_i^T$. Thus, we can let

$$B_i^T = \tilde{F}|_{\tilde{t}_j \times t_i} U_i.$$

This is done be nested multiplication involving orthonormal bases matrices.

As an example, we construct a block 4×4 HSS matrix (2.1) with (2.2) satisfied. After the first stage, we have

$$\begin{pmatrix} \begin{pmatrix} D_1 & U_1 \tilde{F}|_{\tilde{t}_1 \times t_2} \\ U_2 \hat{F}|_{\tilde{t}_2 \times t_1} & D_2 \end{pmatrix} & \begin{pmatrix} U_1 R_1 \\ U_2 R_2 \end{pmatrix} \begin{pmatrix} \tilde{F}|_{\tilde{t}_3 \times t_4} & \tilde{F}|_{\tilde{t}_3 \times t_5} \end{pmatrix} \\ \begin{pmatrix} U_4 R_4 \\ U_5 R_5 \end{pmatrix} \begin{pmatrix} \tilde{F}|_{\tilde{t}_6 \times t_1} & \tilde{F}|_{\tilde{t}_6 \times t_1} \end{pmatrix} & \begin{pmatrix} D_4 & U_4 \tilde{F}|_{\tilde{t}_4 \times t_5} \\ U_5 \tilde{F}|_{\tilde{t}_5 \times t_4} & D_5 \end{pmatrix} \end{pmatrix}.$$

In the second stage, set

$$\begin{aligned} B_1^T &= \tilde{F}|_{\tilde{t}_2 \times t_1} U_1, \ B_4 = \tilde{F}|_{\tilde{t}_5 \times t_4} U_4, \\ B_3^T &= \begin{pmatrix} \tilde{F}|_{\tilde{t}_6 \times t_1} & \tilde{F}|_{\tilde{t}_6 \times t_1} \end{pmatrix} \begin{pmatrix} U_1 & \\ & U_2 \end{pmatrix} \begin{pmatrix} R_1 \\ R_2 \end{pmatrix}. \end{aligned}$$

Assume the HSS rank is bounded by r and the bottom level HSS block row size is $O(r)$, then the total construction cost is $O(rn^2)$, just like the algorithm in [**?**].

2.3.2. Improved ULV-type HSS factorization. The generalized HSS Cholesky factorization in [16] computes an explicit ULV-type factorization of an HSS matrix F. The basic idea is similar to the implicit ULV solution in [11]. That is, first introduce zeros into the off-diagonal, and then use QR factorizations to partially factorize the diagonal block, and then merge the remaining information.

However, for SPD matrices, as pointed out in [16], an improvement can be made so as to avoid dense products of square matrices. The main steps are illustrated in Figure 2.3 and explained as follows. If node i is a leaf, compute a Cholesky factorization $D_i = L_i L_i^T$, and then compute a QL factorization of $L_i^{-1} U_i$ as

$$L_i^{-1} U_i = Q_i \begin{pmatrix} 0 \\ \tilde{U}_i \end{pmatrix} \begin{matrix} m-r \\ r \end{matrix}. \tag{2.5}$$

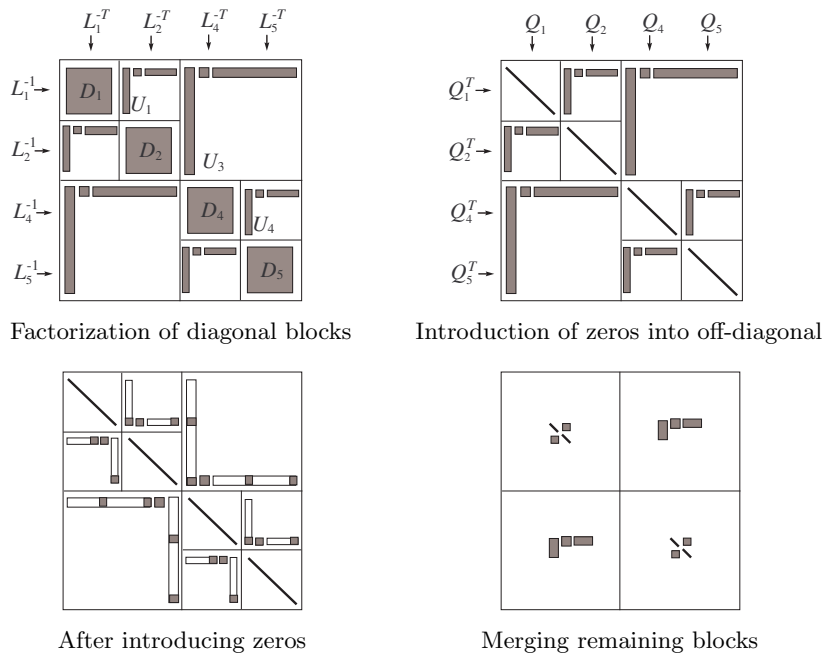
If node i is a non-leaf node with its children c_1 and c_2 being leaves, assume the previous operations are applied to both c_1 and c_2. That is,

$$\begin{aligned} &\begin{pmatrix} Q_{c_1}^T L_{c_1}^{-1} & \\ & Q_{c_2}^T L_{c_2}^{-1} \end{pmatrix} \begin{pmatrix} \bar{D}_{c_1} & \bar{U}_{c_1} B_{c_1} \bar{U}_{c_2}^T \\ \bar{U}_{c_2} B_{c_1}^T \bar{U}_{c_1}^T & \bar{D}_{c_2} \end{pmatrix} \begin{pmatrix} L_{c_1}^{-T} Q_{c_1} & \\ & L_{c_2}^{-T} Q_{c_2} \end{pmatrix} \\ &= \begin{pmatrix} I & \begin{pmatrix} 0 & 0 \\ 0 & \tilde{U}_{c_1} B_{c_1} \tilde{U}_{c_2}^T \end{pmatrix} \\ \begin{pmatrix} 0 & 0 \\ 0 & \tilde{U}_{c_2} B_{c_1}^T \tilde{U}_{c_1}^T \end{pmatrix} & I \end{pmatrix}. \end{aligned} \tag{2.6}$$

Then we merge appropriate blocks on the right-hand side of (2.6) and remove c_1 and c_2 from the HSS tree so that i becomes a leaf with D and U generators

$$\bar{D}_i = \begin{pmatrix} I & \tilde{U}_{c_1} B_{c_1} \tilde{U}_{c_2}^T \\ \tilde{U}_{c_2} B_{c_1}^T \tilde{U}_{c_1}^T & I \end{pmatrix}, \ \bar{U}_i = \begin{pmatrix} \tilde{U}_{c_1} R_{c_1} \\ \tilde{U}_{c_2} R_{c_2} \end{pmatrix}. \tag{2.7}$$

F is reduced to a new matrix called a *reduced HSS matrix*. The same operations then apply recursively. At the root level, a direct Cholesky factorization is computed. Figure 2.4 shows how an HSS tree is reduced along the factorization.



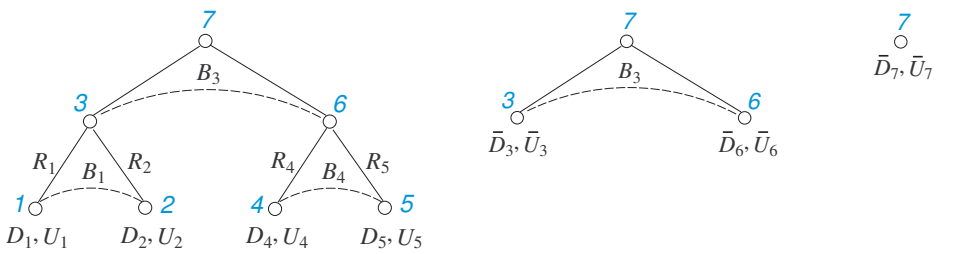
Factorization of diagonal blocks

Introduction of zeros into off-diagonal

After introducing zeros

Merging remaining blocks

FIG. 2.3. *An improved generalized HSS Cholesky factorization scheme.*



(i) HSS tree (ii) After eliminating one level (iii) Root

FIG. 2.4. *How the HSS tree is reduced along the generalized HSS Cholesky factorization.*

Note that due to the special form of $\bar{D}_i$ in (2.7), the intermediate Cholesky factorization $\bar{D}_i = L_i L_i^T$ at upper levels are done quickly. That is, we have

$$L_i = \begin{pmatrix} I & \\ \tilde{U}_{c_2} B_{c_1}^T \tilde{U}_{c_1}^T & \hat{L}_i \end{pmatrix}, \ \tilde{L}_i \tilde{L}_i^T = I - \tilde{U}_{c_2} B_{c_1}^T (\tilde{U}_{c_2} B_{c_1}^T)^T.$$

Similarly, the update $L_p^{-1}\tilde{U}_p$ can also be done quickly as

$$L_i^{-1}\tilde{U}_i = \begin{pmatrix} \tilde{U}_{c_1}R_{c_1} \\ \tilde{L}_i^{-1}(\tilde{U}_{c_2}R_{c_2} - \tilde{U}_{c_2}B_{c_1}^T\tilde{U}_{c_1}^T\tilde{U}_{c_1}R_{c_1}) \end{pmatrix} = \begin{pmatrix} \tilde{U}_iR_i \\ \tilde{L}_i^{-1}\tilde{U}_{c_2}(R_{c_2} - B_{c_1}^T\tilde{U}_{c_1}^T\bar{U}_{c_1}R_{c_1}) \end{pmatrix}.$$

This improved algorithm is thus a lot more efficient than the basic algorithms in [16], which needs two dense square matrix products in the form of $Q_i^T\tilde{D}_iQ_i$ and a partial QR factorization of a full diagonal block $\tilde{D}_i$ for each node i.

We summarize the algorithm as follows.

Algorithm 1. (Improved ULV-type HSS factorization)
`subroutine hss_ulv`(H)
`for` all nodes i at the bottom level $l_{\max}$, let $\bar{U}_i = U_i$, $\bar{D}_i = D_i$.
`for` $l = l_{\max}, l_{\max}-1, \ldots, 1$ `do`
  `for` all nodes i at level $l-1$ `do`
  1. (Factorization) `for` each child c of i, compute $\bar{D}_c = L_cL_c^T$ (Cholesky).
  2. (Compression) Compute a QR factorization $L_c^{-1}\bar{U}_c = Q_c\begin{pmatrix} 0 \\ \tilde{U}_c \end{pmatrix}$.
  3. (Merge) Form $\bar{D}_i$ and $\bar{U}_i$ as in (2.7)
  `end for`
`end for`
Compute the Cholesky factorization $\bar{D}_k = L_kL_k^T$.
`end subroutine`

Such an algorithm computes a ULV-type HSS factorization

$$H = \hat{\mathbf{L}}\hat{\mathbf{L}}^T, \tag{2.8}$$

where $\hat{\mathbf{L}}$ is given by a sequence of orthogonal and lower triangular matrices. We call $\hat{\mathbf{L}}$ a generalized HSS Cholesky factor, which can be represented recursively. Let $H^{(l)}, l = l_{\max}, l_{\max}-1, \ldots, 1, 0$ be the reduced matrices at different levels, where $H^{(l_{\max})} \equiv H$, and $H^{(l)}$ is a reduced matrix obtained from $H^{(l+1)}$ after the elimination of all nodes at level $l+1$. Assume the nodes at level l are $j_1, j_2, \ldots, j_\alpha$. Also let P_l be a permutation matrix which performs all the merge steps during the elimination of level $l+1$ and also forms the new reduced HSS matrix. That is,

$$P_lX_l\mathrm{diag}\left(\begin{pmatrix}\bar{U}_{c_{j_1;1}}R_{c_{j_1;1}} \\ \bar{U}_{c_{j_1;2}}R_{c_{j_1;2}}\end{pmatrix}, \ldots, \begin{pmatrix}\bar{U}_{c_{j_\alpha;1}}R_{c_{j_\alpha;1}} \\ \bar{U}_{c_{j_\alpha;2}}R_{c_{j_\alpha;2}}\end{pmatrix}\right) = \begin{pmatrix} 0 \\ \mathrm{diag}\left(\bar{U}_{j_1}, \ldots, \bar{U}_{j_\alpha}\right)\end{pmatrix}, \tag{2.9}$$

$$P_lX_lH^{(l+1)}X_l^TP_l^T = \begin{pmatrix} I & \\ & H^{(l)} \end{pmatrix}, \tag{2.10}$$

where

$$X_l = \mathrm{diag}\left(\begin{pmatrix} Q_{c_{j_1;1}}^TL_{c_{j_1;1}}^{-1} & \\ & Q_{c_{j_1;2}}^TL_{c_{j_1;2}}^{-1}\end{pmatrix}, \ldots, \begin{pmatrix} Q_{c_{j_\alpha;1}}^TL_{c_{j_\alpha;1}}^{-1} & \\ & Q_{c_{j_\alpha;2}}^TL_{c_{j_\alpha;2}}^{-1}\end{pmatrix}\right).$$

Then we let $\hat{\mathbf{L}} \equiv \hat{\mathbf{L}}^{(l_{\max})}$, where

$$\hat{\mathbf{L}}^{(l+1)} = X_l^{-1}P_l^T\begin{pmatrix} I & \\ & \hat{\mathbf{L}}^{(l)}\end{pmatrix}P_l,\ l = l_{\max}-1, l_{\max}-2, \ldots, 1, 0, \tag{2.11}$$

and $\hat{\mathbf{L}}^{(0)}$ is the Cholesky factor associated with the root level. Here, $H^{(l)} = \hat{\mathbf{L}}^{(l)}(\hat{\mathbf{L}}^{(l)})^T$.

$\hat{\mathbf{L}}$ is formed by the set of matrices $\{L_i, Q_i, P_l\}$ associated with the nodes i and levels l.

2.3.3. Improved ULV-type HSS solution. Due to the improved ULV-type HSS factorization, the solution procedure with the generalized HSS Cholesky factor is much simpler than the one in [49]. For example, we consider the solution of the following system by forward substitution:

$$\hat{\mathbf{L}}y = b.$$

Based on the HSS tree of H, we introduce a solution tree, where each node i is associated with b_i, a piece of b, and y_i, a piece of y. These pieces are defined recursively. First, b is partitioned into b_i pieces according to the leaf level L_i sizes. For each leaf i, let

$$\hat{b}_i = Q_i^T L_i^{-1} b_i.$$

Partition $\hat{b}_i$ conformably following the right-hand side of (2.5) as $\hat{b}_i = \begin{pmatrix} \hat{b}_{i,1} \\ \hat{b}_{i,2} \end{pmatrix}$. Then set $y_i = \hat{b}_{i,1}$. Assume similar operations are also conducted the sibling node j of i, where $j > i$. Then set

$$b_p = \begin{pmatrix} \hat{b}_{i,2} \\ \hat{b}_{j,2} \end{pmatrix},$$

where p is the parent of i and j. Similar operations then apply to non-leaf nodes. These steps proceed until the root node k is visited, where

$$y_k = L_k^{-1} b_k. \tag{2.12}$$

Then all these solution pieces y_i form the solution y.

Algorithm 2. (Improved ULV-type low-triangular HSS solution)
`subroutine` `l_hsssol`$(\hat{\mathbf{L}}, b)$
`for` all nodes i at the bottom level $l_{\max}$, partition b into pieces b_i.
`for` $l = l_{\max}, l_{\max} - 1, \ldots, 1$ `do`
  `for` all nodes i at level $l - 1$ `do`
    `for` each child $c = c_1, c_2$ of i, compute $\hat{b}_c = Q_c^T L_c^{-1} b_c \equiv \begin{pmatrix} y_c \\ \hat{b}_{c,2} \end{pmatrix}$.
    $b_i = \begin{pmatrix} \hat{b}_{c_1,2} \\ \hat{b}_{c_2,2} \end{pmatrix}$.
  `end for`
`end for`
Compute $y_k = L_k^{-1} b_k$.
`end subroutine`

Assume the solution pieces provided at each level l form a vector $\bar{y}^{(l)}$. Then the solution y is formed by collection all $\bar{y}^{(l)}$ with appropriate permutations (Figure 2.5). That is, we define

$$y^{(l)} = P_l^T \begin{pmatrix} \bar{y}^{(l)} \\ y^{(l-1)} \end{pmatrix}, \ l = l_{\max}, l_{\max} - 1, \ldots, 2, 1 \tag{2.13}$$

with $y^{(0)} = y_k$. Then $y \equiv y^{(l_{\max})}$.

The backward substitution procedure can be similarly considered.

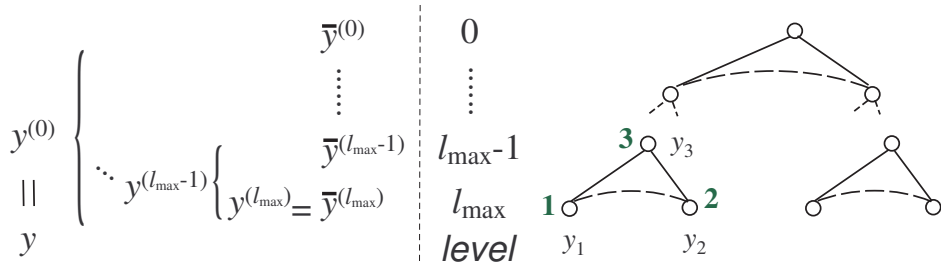


FIG. 2.5. *Solution pieces generated in Algorithm 2. To save space, the solution pieces are drawn as horizontal bars (transposed forms) instead of vertical ones.*

3. Relaxing rank requirements in HSS operations. Existing HSS algorithms generally assume the HSS ranks to be bounded. Motivated by the concept of rank functions in [20], we relax the rank requirement in HSS operations. For simplicity, we assume the HSS tree is a perfect binary tree, and all HSS block rows at the same level have the same row sizes. Also assume the bottom HSS block rows have row dimensions m_0 so that $l_{\max} = O(\log_2 \frac{n}{m_0})$.

At level l, the HSS block rows have row dimensions $m_{l_{\max}-l} = m_0 2^{l_{\max}-l}$. Instead of requiring their numerical ranks to be bounded by a constant, we allow them to increase with the block size and assume them to be bounded by

$$r_l = (l_{\max} - l + 1)m_0 = (\log_2 \frac{n}{m_0} - l + 1)m_0.$$

Clearly, r_l increases as n_l increases (or l decreases).

For HSS constructions algorithms such as the one in Section 2.3.1, the cost associated with each tree node at level l is $O(r_l^2 n)$ flops. There are 2^l nodes at level l. Using the facts that $\sum_{l=0}^{l_{\max}} (l_{\max} - l + 1)^2 2^l \approx 12 \times 2^{l_{\max}}$ and $2^{l_{\max}} m_0 = n$, we have the total cost for the HSS construction

$$\sum_{l=0}^{l_{\max}} O(r_l^2 n) 2^l = O(m_0^2 n \sum_{l=0}^{l_{\max}} (l_{\max} - l + 1)^2 2^l) = O(m_0 n^2).$$

Thus, after relaxing the rank requirement, we do not increase the complexity.

Similarly, for HSS algorithms such as the ULV factorization Algorithm 1, the cost associated with each tree node is $O(r(l)^3)$ flops. Using $\sum_{l=0}^{l_{\max}} (l_{\max} - l + 1)^3 2^l) \approx 52 \times 2^{l_{\max}}$ we have the total cost

$$\sum_{l=0}^{l_{\max}} O(r_l^3) 2^l = O(m_0^3 \sum_{l=0}^{l_{\max}} (l_{\max} - l + 1)^3 2^l) = O(m_0^2 n).$$

In these situations, $r_l = O(\log_2 m_{l_{\max}-l})$, and the HSS rank is bounded by $O(\log_2 n)$. More generally, we can allow r_l to be

$$r_l = (\log_2 m_{l_{\max}-l})^p = m_0^p (\log_2 \frac{n}{m_0} - l + 1)^p,$$

where p is a positive integer. For example, when $p = 2$ we have

$$\sum_{l=0}^{l_{\max}} m_0^6 (l_{\max} - l + 1)^6 2^{l_{\max}} \approx 18\,732 \times m_0^6 2^{l_{\max}} = O(m_0^5 n).$$

Thus, the ULV factorization still costs $O(n)$ (with a larger constant).

Similarly, the storage remains $O(n)$ under the new rank conditions.

We can further relax the rank requirements so that r_l increases as $O(m_l^{1/p})$, where p is a positive integer. The derivations are similar. For example, for $p = 3$ and 2, the HSS construction costs are now

$$\sum_{l=0}^{l_{\max}} O\left(m_{l_{\max}-l}^{1/3}\right)^2 n2^l = \sum_{l=0}^{L} O\left(\left(m_0 \frac{n}{2^l}\right)^{2/3}\right) n2^l = O(m_0^{1/3} n^2),$$
$$\sum_{l=0}^{l_{\max}} O\left(m_{l_{\max}-l}^{1/2}\right)^2 n2^l = \sum_{l=0}^{L} O\left(m_0 \frac{n}{2^l}\right) n2^l = O(m_0 n^2 \log_2 n),$$

respectively.

All these results are summarized in the following theorem, with m_0 removed from the counts.

THEOREM 3.1. *Suppose an order n matrix F is recursively partitioned into $l_{\max} = O(\log_2 \frac{n}{m_0})$ levels of HSS blocks following a perfect binary tree, so that the bottom level HSS block row size is m_0. Let $m_{l_{\max}-l} = O(m_0 2^{l_{\max}-l})$ be the row dimensions of the HSS block rows at level l, and r_l be their maximum numerical ranks.*

1. *When $r_l = O((\log_2 m_{l_{\max}-l})^p)$ (so that $\max_l r_l = O((\log_2 n)^p)$) with a non-negative integer p, an HSS form for F can be constructed in ξ_c flops, a ULV factorization of the HSS form as in Algorithm 1 needs ξ_s flops, and the HSS form or its ULV factorized form can be stored in σ space, where*

$$\xi_c = O(n^2),\ \xi_s = O(n),\ \sigma = O(n).$$

   (These are the same counts as in the classical situation [11, 16, etc.] where $r_l = O(r_0)$ or $c = 0$.)
2. *When $r_l = O(m_{l_{\max}-l}^{1/p})$ (so that $\max_l r_l = O(n^{1/p})$), where p is a positive integer:*
   (a) If $p > 3$, then the counts are

$$\xi_c = O(n^2),\ \xi_s = O(n),\ \sigma = O(n).$$

   (b) If $p = 3$, then the counts are

$$\xi_c = O(n^2),\ \xi_s = O(n \log_2 n),\ \sigma = O(n).$$

   (c) If $p = 2$, then the counts are

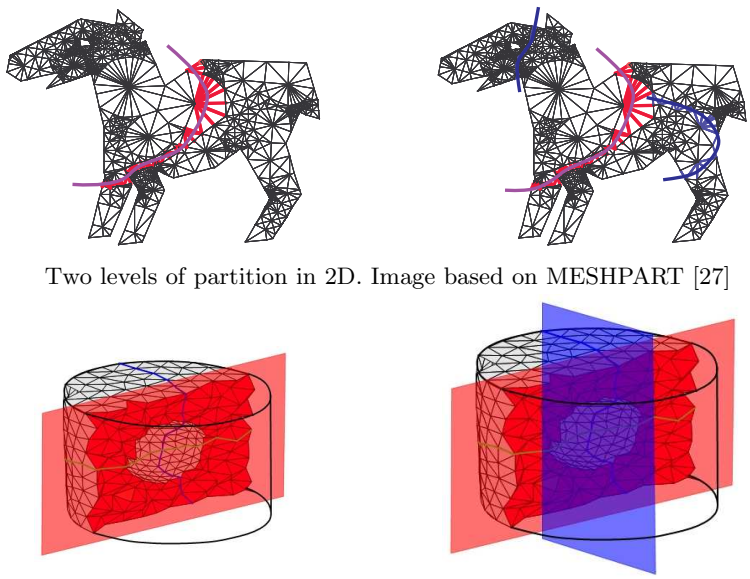
$$\xi_c = O(n^2 \log_2 n),\ \xi_s = O(n^{3/2}),\ \sigma = O(n \log_2 n).$$

These results can also be generalized to other structured matrices such as SSS matrices, quasiseparable matrices, and $\mathcal{H}^2$ matrices.

4. Structured supernodal multifrontal factorization. In this section, we consider the factorization of sparse discretized matrices. The HSS algorithms in the previous sections are applied to the dense intermediate matrices in a multifrontal factorization framework.

4.1. Review of nested dissection ordering and multifrontal method.

4.1.1. Nested dissection ordering. Consider the mesh used in the discretization as an undirected graph. Each vertex i corresponds to a row and a column of A, and each edge (i, j) corresponds to entries $a_{ij} = a_{ji} \neq 0$. Nested dissection recursively divides the mesh into subregions with separators [26]. At the first level, a separator divides the entire grid into two subregions (plus the separator itself). The two subregions are further divided recursively. See Figure 4.1.



Two levels of partition in 2D. Image based on MESHPART [27]

Three levels of partition in 3D. Image based on DISTMESH [44]

FIG. 4.1. *Nested dissection ordering for irregular meshes.*

Lower level separators are ordered before upper level ones. For example, Figure 4.2 shows the reordering of the mesh and nonzero pattern of the corresponding discretized matrix A after reordering. During the factorization of A, the elimination of a mesh point mutually connects points which are previously connected to it [43, 47]. This creates fill-in. For 2D problems, nest dissection leads to $O(n^{1.5})$ complexity factorizations with $O(n \log n)$ storage, which are essentially lower bounds for the exact factorization with any ordering [34] (ignoring special techniques such as Strassen's algorithm).

4.1.2. Multifrontal method. The multifrontal method reorganizes the factorization $A = \mathcal{L}\mathcal{L}^T$ into partial factorizations of intermediate dense matrices [24, 37]. The factorization is conducted following a tree called elimination tree, or more generally, an assembly tree. In general, an elimination tree $\mathcal{T}$ has n nodes and a node p is the parent of i if and only if

$$p = \min\{j > i | l_{ji} \neq 0\}.$$

Use $\mathcal{T}[i]$ to denote the subtree of $\mathcal{T}$ with root i. In the elimination, the contributions from child nodes are passed to parents in terms of outer products. Let $\{s_0, s_1, \ldots, s_d\}$ be the row indices of nonzeros in $\mathcal{L}_{:,i}$, the i-th column of $\mathcal{L}$ with $s_0 = i$. Define the

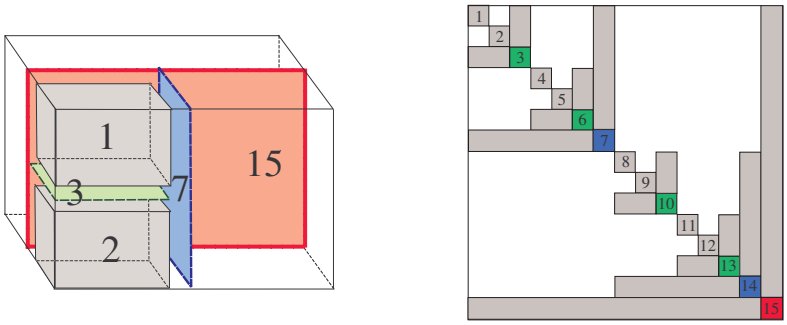


FIG. 4.2. *Nonzero pattern of A after the nested dissection ordering of the rows and columns.*

subtree update matrix corresponding to i as

$$\bar{\mathcal{U}}_i = -\sum_{j\in\mathcal{T}[i]\setminus i} \mathcal{L}_{\{s_0,s_1,\ldots,s_d\},j}\mathcal{L}^T_{\{s_0,s_1,\ldots,s_d\},j},$$

The i-th *frontal matrix* is defined to be

$$\mathcal{F}_i = \begin{pmatrix} a_{ii} & A^T_{\{s_0,s_1,\ldots,s_d\},i} \\ A_{\{s_0,s_1,\ldots,s_d\},i} & 0 \end{pmatrix} + \bar{\mathcal{U}}_i.$$

One step of elimination applied to F_i provides the column $\mathcal{L}_{\{s_0,s_1,\ldots,s_d\},i}$

$$\mathcal{F}_i = \begin{pmatrix} l_{ii} & 0 \\ \mathcal{L}_{\{s_1,s_2,\ldots,s_d\},i} & I \end{pmatrix} \begin{pmatrix} l_{ii} & \mathcal{L}^T_{\{s_1,s_2,\ldots,s_d\},i} \\ 0 & \mathcal{U}_i \end{pmatrix},$$

where $\mathcal{U}_i$ is the contribution from the $T[i]$ to the parent of i and is called the i-th *update matrix*. U_i has the form

$$\mathcal{U}_i = -\sum_{j\in\mathcal{T}[i]} \mathcal{L}_{\{s_1,s_2,\ldots,s_d\},j}\mathcal{L}^T_{\{s_1,s_2,\ldots,s_d\},j}.$$

In the multifrontal method, lower level frontal matrices are formed and partially eliminated to provide some columns of L and the update matrices. The update matrices are then used to form upper level partial update matrices and thus upper level frontal matrices. This process is called *extend-add*, denoted

$$\mathcal{F}_i = \begin{pmatrix} a_{j,j} & A^T_{\{s_1,s_2,\ldots,s_d\},i} \\ A_{\{s_1,s_2,\ldots,s_d\},i} & 0 \end{pmatrix} \Updownarrow \mathcal{U}_{c_1} \Updownarrow \mathcal{U}_{c_2} \Updownarrow \cdots \Updownarrow \mathcal{U}_{c_q},$$

where nodes $c_1, c_2, \ldots, c_q$ are the children of j in the elimination tree. The process then repeats along the elimination tree.

4.2. Structured supernodal multifrontal factorizations. We use nested dissection to reorder A and then form a binary assembly tree where each separator is treated as a node in the tree. Without loss of generality, we can use a full binary tree, since for any specified number of leaves, there always exists a full binary tree [16]. A supernodal version multifrontal method is then used to eliminate the separators. Unlike the algorithm in [49] for regular meshes, here we allow the mesh to be irregular

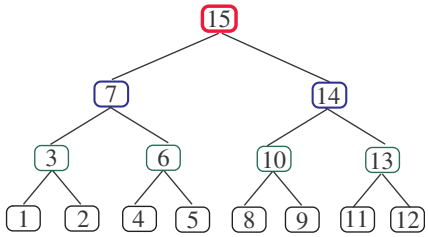


FIG. 4.3. *Supernodal version elimination tree for the problem in Figure 4.2.*

and the separators to be more general, as illustrated in Figure 4.1. Figure 4.3 shows the assembly tree for the mesh in Figure 4.2.

Let $\{s_1, s_2, \ldots, s_\gamma\}$ be the set of neighbor separators of separator i at the same or upper levels of i. Assume the index set of separator s_j in A is t_{s_j}. The elimination of a separator mutually connects nodes that are connected to it [32, 43]. For notational convenience, for each neighbor i_j, let $\tilde{t}_{s_j}$ be the subset of t_{i_j} that is connected to separator i due to lower level eliminations. Also let c_1 and c_2 be the children of i. The frontal matrix $\mathcal{F}_i$ is formed by the block form extend-add operation

$$\mathcal{F}_i = \mathcal{F}_i^0 \updownarrow \mathcal{U}_{c_1} \updownarrow \mathcal{U}_{c_2}, \ \mathcal{F}_i^0 \equiv \begin{pmatrix} A|_{t_i \times t_i} & \left(A|_{(\cup_{j=1}^{\gamma} \tilde{t}_{s_j}) \times t_i}\right)^T \\ A|_{(\cup_{j=1}^{\gamma} \tilde{t}_{s_j}) \times t_i} & 0 \end{pmatrix} \tag{4.1}$$

For convenience, we assume $\mathcal{F}_i$ to have the following form:

$$\mathcal{F}_i \equiv \begin{pmatrix} F_{i,i} & F^T_{\{s_1,s_2,\ldots,s_\gamma\},i} \\ F_{\{s_1,s_2,\ldots,s_\gamma\},i} & F_{\{s_1,s_2,\ldots,s_\gamma\},\{s_1,s_2,\ldots,s_\gamma\}} \end{pmatrix}. \tag{4.2}$$

In our structured multifrontal method, separator i is eliminated by factorizing $F_{i,i}$ into structured forms with the methods in section 2. $F_{\{s_1,s_2,\ldots,s_\gamma\},i}$ is also compressed in the meantime, so that the Schur complement of $F_{i,i}$ (the update matrix $\mathcal{U}_i$) can be computed by a low-rank update. Then $\mathcal{U}_i$ participates in the extend-add operation of the parent frontal matrix, as in the standard supernodal multifrontal method. Depending on the dense structured methods for factorizing $F_{i,i}$, we also have two types of sparse factorizations.

4.2.1. Robust structured multifrontal factorization. In this scheme, we use the robust HSS Cholesky factorization method in section 2.2 to partially factorize the frontal matrices. That is, for a node i of the elimination tree, its frontal matrix $\mathcal{F}_j$ in (4.1) is (approximately) factorized as

$$\mathcal{F}_i \approx \begin{pmatrix} \mathbf{L}_{i,i} & \\ \mathbf{L}_{\{s_1,s_2,\ldots,s_\gamma\},i} & I \end{pmatrix} \begin{pmatrix} I & \\ & \mathcal{U}_i \end{pmatrix} \begin{pmatrix} \mathbf{L}_{i,i}^T & \mathbf{L}^T_{\{s_1,s_2,\ldots,s_\gamma\},i} \\ & I \end{pmatrix}, \tag{4.3}$$

where $\mathbf{L}_{i,i}$ is a low-triangular HSS matrix, $\mathbf{L}_{\{s_1,s_2,\ldots,s_\gamma\},i}$ is in structured form obtained in the factorization of $F_{j,j}$, and $\mathcal{U}_j$ is dense. This partial factorization can be done as follows.

Set a full HSS tree T with $k+2$ nodes where the left and right children of the root are k and $k+1$ respectively. See Figure 4.4. The subtree $T[k]$ is used as the HSS tree for completely factorizing $F_{i,i}$, and the single node $k+1$ is for the unfactorized part $F_{\{s_1,s_2,\ldots,s_\gamma\},\{s_1,s_2,\ldots,s_\gamma\}}$ of $\mathcal{F}_i$. The algorithm in [50] is applied to $\mathcal{F}_i$ following

the postordering traversal of the nodes of the tree T. The factorization stops after the entire $T[k]$ is visited (or after the Schur complement $\mathcal{U}_i$ is computed). At the point, $F_{i,i} \approx \mathbf{L}_{i,i}\mathbf{L}_{i,i}^T$ where $\mathbf{L}_{i,i}$ is an HSS matrix. Assume $\mathbf{L}_{i,i}$ has generators D_i, U_i, R_i. With the multi-level HSS representation as in Figure 2.2 we have a structured form

$$\mathbf{L}_{i,i} = \left(\begin{array}{c|c} \left(\begin{array}{c|c} \ddots & 0 \\ \hline U_{c_{k,1,2}} B_{c_{k,1,2}} U_{c_{k,1,1}}^T & \ddots \end{array}\right) & 0 \\ \hline U_{c_{k,2}} B_{c_{k,2}} U_{c_{k,1}}^T & \left(\begin{array}{c|c} \ddots & 0 \\ \hline U_{c_{k,2,2}} B_{c_{k,2,2}} U_{c_{k,2,1}}^T & \ddots \end{array}\right) \end{array}\right), \tag{4.4}$$

$$\mathbf{L}_{\{s_1,s_2,\ldots,i_d\},i} = U_{k+1} B_k^T U_k^T, \tag{4.5}$$

where $c_{k,1}, c_{k,2}, \ldots$ are the children of appropriate nodes as shown in Figure 4.4. Note that U_{k+1} and B_k are explicitly available, and U_k is implicitly represented by lower level U and R generators, and have orthonormal columns. Thus, $\mathcal{U}_i$ can be formed explicitly with a low-rank update

$$\mathcal{U}_i = F_{\{s_1,s_2,\ldots,s_\gamma\},\{s_1,s_2,\ldots,s_\gamma\}} - U_{k+1} B_k^T B_k U_{k+1}^T, \tag{4.6}$$

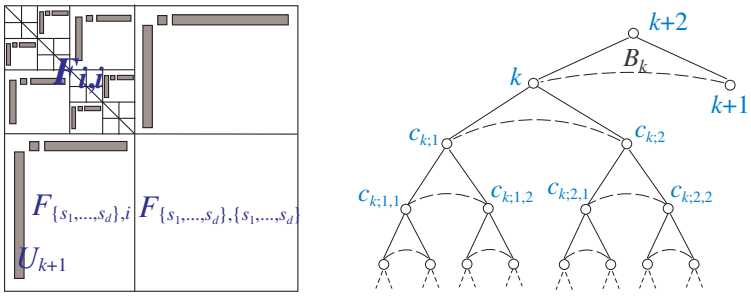


FIG. 4.4. *Partial factorization of a frontal matrix F_j and the HSS tree used. F_j is shown with its leading block $F_{j,j}$ represented by structured factors. The subtree $T[k-2]$ is for $F_{j,j}$.*

According to the idea of Schur compensation in [50], $\mathcal{U}_i$ is equal to the exact Schur complement of $F_{i,i}$ plus a positive definite term, and is guaranteed to be positive definite. The approximate update matrix $\mathcal{U}_i$ then participate in the construction of the frontal matrix of the parent node just like in the standard multifrontal method. The parent frontal matrix is then also guaranteed to be positive definite.

This process then proceeds along the elimination tree. After the elimination, we have an approximate factor $\mathcal{L}$ of A in structured form. See Figure 4.5 for an example. $\mathcal{L}$ is associated with two layers of postordering trees, the outer layer assembly tree, and an inner layer HSS tree for each node of the elimination tree. A node i of the assembly tree is associated with an HSS form $\mathbf{L}_{i,i}$ and also the U and B generators in (4.6). These are used in the structured multifrontal solution.

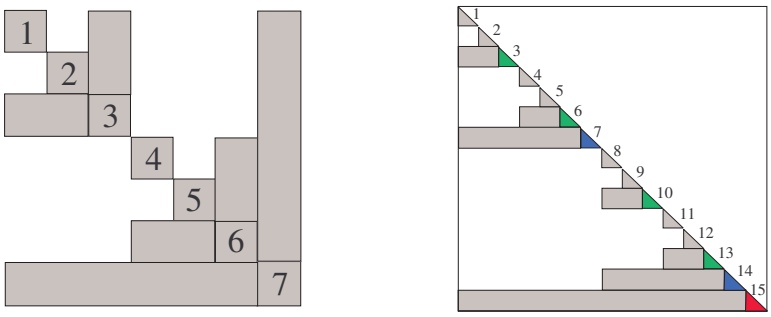


FIG. 4.5. *A sparse matrix in nested dissection ordering, and its structured factor.*

4.2.2. Scalable structured multifrontal factorization. Another approach is to use the scalable algorithms in section 2.3. We first construction an HSS approximation to (4.2). An HSS tree similar to the one in Figure 4.4 is used. Assume the HSS form is

$$\mathcal{F}_i \approx \begin{pmatrix} H & U_k B_k U_{k+1}^T \\ U_{k+1} B_k^T U_k^T & D_{k+1} \end{pmatrix}, \tag{4.7}$$

where H is an HSS approximation to $F_{i,i}$, and $D_{k+1} \equiv F_{\{s_1,s_2,\ldots,s_d\},\{s_1,s_2,\ldots,i_d\}}$. Then the ULV-type HSS factorization is applied to H. Assume $H = \hat{\mathbf{L}}_i \hat{\mathbf{L}}_i^T$ is obtained. Then

$$\mathcal{F}_i \approx \begin{pmatrix} \hat{\mathbf{L}}_i & \\ U_{k+1} B_k^T U_k^T \hat{\mathbf{L}}_i^{-T} & I \end{pmatrix} \begin{pmatrix} I & \\ & \mathcal{U}_i \end{pmatrix} \begin{pmatrix} \hat{\mathbf{L}}_i^T & \hat{\mathbf{L}}_i^{-1} U_k B_k U_{k+1}^T \\ & I \end{pmatrix}, \tag{4.8}$$

where

$$\mathcal{U}_i = D_{k+1} - \left(U_{k+1} B_k^T U_k^T\right) H^{-1} \left(U_k B_k U_{k+1}^T\right). \tag{4.9}$$

We emphasize that we do not need to explicitly apply $\hat{\mathbf{L}}_i^{-1}$ to other matrices (or to store $U_{k+1} B_k^T U_k^T \hat{\mathbf{L}}_i^{-T}$), neither do we compute $\mathcal{U}_i$ by explicitly forming H^{-1}. The first issue is addressed in Theorem **??** below, and the second issue is elaborated as follows.

After the ULV-type HSS factorization of H, the HSS tree of $\mathcal{F}_i$ is reduced to the one in Figure 4.6, following the tree reduction as illustrated in Figure 2.4. That is, the subtree $T[k]$ is reduced to a single node with associated generators $\bar{D}_k$ and $\bar{U}_k$ are given by (2.7) with $i \equiv k$.

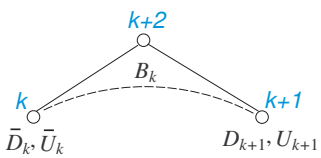


FIG. 4.6. *After the generalized HSS Cholesky factorization of the leading block H in (4.7).*

This process does not involve explicit Schur complement computation and is scalable. On the other hand, the update matrix (as a Schur complement) $\mathcal{U}_i$ is not

directly available. However, we can still compute $\mathcal{U}_i$ conveniently with the explicit ULV- factorization of H. Using the notation in the section 2.3, we have the following results.

THEOREM 4.1. *Assume Algorithm 1 is applied to H in (4.7). Then*

$$U_k^T H^{-1} U_k = \bar{U}_k^T \bar{D}_k^{-1} \bar{U}_k, \tag{4.10}$$

where $\bar{D}_k$ and $\bar{U}_k$ are given by (2.7) after the visit of node $i \equiv k$. Thus,

$$\mathcal{U}_i = D_{k+1} - \left(U_{k+1} B_k^T \bar{U}_k^T\right) \bar{D}_k^{-1} \left(\bar{U}_k B_k U_{k+1}^T\right). \tag{4.11}$$

Proof. Assume $T[k]$ (the HSS tree of H) has $l_{\max}$ levels so that node k is at level 0. After eliminating the nodes at level $l_{\max}$, the same operations as in (2.9) also transforms U_k to a new matrix. Thus, we let $U_k^{(l_{\max})} \equiv U_k$, and define the intermediate matrix $U_k^{(l)}$ recursively as follows. After the elimination of all the nodes at level $l+1$, the matrix $U_k^{(l+1)}$ is transformed into a reduced form $U_k^{(l)}$, and the tree has U generators $\bar{U}_{j_1}, \ldots, \bar{U}_{j_s}$ at level l as in (2.9). That is,

$$P_l X_l U_k^{(l+1)} = \begin{pmatrix} 0 \\ U_k^{(l)} \end{pmatrix}.$$

For $l = l_{\max} - 1, l_{\max} - 2, \ldots, 1$, we have the following recursive relationship:

$$\begin{aligned}
&\left(U_k^{(l+1)}\right)^T \left(H^{(l+1)}\right)^{-1} U_k^{(l+1)} \\
&= \left(U_k^{(l+1)}\right)^T \left((P_l X_l)^{-1} \begin{pmatrix} I & \\ & H^{(l)} \end{pmatrix} (X_l^T P_l^T)^{-1}\right)^{-1} P_l X_l U_k^{(l+1)} \\
&= \left(U_k^{(l+1)}\right)^T X_l^T P_l^T \begin{pmatrix} I & \\ & \left(H^{(l)}\right)^{-1} \end{pmatrix} P_l X_l U_k^{(l+1)} \\
&= \begin{pmatrix} 0 & \left(U_k^{(l)}\right)^T \end{pmatrix} \begin{pmatrix} I & \\ & \left(H^{(l)}\right)^{-1} \end{pmatrix} \begin{pmatrix} 0 \\ U_k^{(l)} \end{pmatrix} \\
&= \left(U_k^{(l)}\right)^T \left(H^{(l)}\right)^{-1} U_k^{(l)}.
\end{aligned}$$

Thus, $(U_k^{(l_{\max})})^T (H^{(l_{\max})})^{-1} U_k^{(l_{\max})} = (U_k^{(0)})^T \left(H^{(0)}\right)^{-1} U_k^{(0)}$. On the other hand, $U_k^{(l_{\max})} = U_k$, $H^{(l_{\max})} = H$, $U_k^{(0)} = \bar{U}_k$, and $H^{(0)} = \bar{D}_k$. Therefore, (4.10) holds.

(4.9) and (4.10) lead to (4.11). Thus, $\mathcal{U}_i$ can be computed quickly with a low-rank update, since $\bar{D}_k$ is a much smaller matrix. ∎

The result can be stated in a slightly more general form.

COROLLARY 4.2. *For each node $i = 1, 2, \ldots, k-2$ of the HSS tree of H,*

$$U_i^T D_i^{-1} U_i = \bar{U}_i^T \bar{D}_i^{-1} \bar{U}_i.$$

Again, update matrices participate in the standard extend-add operation to form upper level frontal matrices. This process then proceeds along the elimination tree. After the elimination, we have a sparse ULV-type factorization $A \approx \mathcal{L}\mathcal{L}^T$.

4.3. Algorithm and performance analysis. In practice, to achieve nearly optimal complexity, the structured factorization only starts from certain switching level $\mathbf{l}_s$. At some bottom levels of the elimination, we use the traditional multifrontal method. This is shown in Theorem 4.3 after the following algorithm. Let $\mathcal{T}$ be the assembly tree of the m separators from nested dissection.

ALGORITHM 3. (*Robust/Scalable structured multifrontal factorizations*)
`subroutine rsmf/ssmf`
`for` nodes (separators) $i = 1, 2, \ldots, m$ of $\mathcal{T}$

1. `if` i is a leaf, form $\mathcal{F}_i \equiv \mathcal{F}_i^0$, where $\mathcal{F}_i^0$ is given in (4.1)
2. `if` i is at level $\mathbf{l} > \mathbf{l}_s$
   (a) Compute traditional Cholesky factorization $F_{i,i} = \hat{\mathbf{L}}_i \hat{\mathbf{L}}_i^T$ of $F_{i,i}$ in (4.2)
   (b) Compute the Schur complement $\mathcal{U}_i$
3. `else`
   (a) Apply the robust HSS Cholesky factorization to $\mathcal{F}_i$ so that (4.4) and (4.5) in (4.3) are computed
   (b) Compute $\mathcal{U}_i$ with a low-rank update as in (4.6)

   } `rsmf` (*robust structured multifrontal*)

   `or`

   (a) Apply the ULV-type HSS factorization to $\mathcal{F}_i$ so that $\hat{\mathbf{L}}_i$ in (4.8) is computed
   (b) Compute $\mathcal{U}_i$ with a low-rank update as in (4.11)

   } `ssmf` (*scalable structured multifrontal*)
4. `if` i is a left node, push $\mathcal{U}_i$ onto the update matrix stack
5. `else`
   (a) Pop $\mathcal{U}_{c_1}$ from the update matrix stack
   (b) $\mathcal{F}_p = \mathcal{F}_p^0 \Updownarrow \mathcal{U}_{c_1} \Updownarrow \mathcal{U}_i$, where p is the parent of i

In a parallel implementation, we can traverse the assembly tree levelwise. The algorithm can be applied to general sparse SPD matrices. But for simplicity, we only consider its complexity in terms of sparse matrices arising from 2D and 3D discretized PDEs.

THEOREM 4.3. *Suppose Algorithm 3 is used to factorize a discretized matrix A on a 2D $n \times n$ regular grid. Assume each frontal matrix $\mathcal{F}_i$ has order n_i and is treated as F in Theorem 3.1 so that the HSS blocks at level l of the HSS tree of $\mathcal{F}_i$ has row dimension $m_{l_{\max}-l}$ and rank r_l.*

(i) If for all separators i we have

$$r_l = O((\log_2 m_{l_{\max}-l})^p),\ p \geq 0\text{: integer (so that } \max_l r_l = O((\log_2 n_i)^p)\text{),}$$

or

$$r_l = O(m_{l_{\max}-l}^{1/p}),\ p \geq 3\text{: integer (so that } \max_l r_l = O(n_i^{1/p})\text{),}$$

then the optimal complexity (ξ) of Algorithm 3 and the corresponding storage (σ) for the structured factor are given by

$$\xi = O(n^2 \log_2 n),\ \sigma = O(n^2 \log_2(\log_2 n)),$$

respectively. In such a situation, the factorization costs before and after the switch level $\mathbf{l}_s$ are equal.

(ii) If for all separators i we have $r_l = O(m_{l_{\max}-l}^{1/2})$ (so that $\max_l r_l = O(n_i^{1/2})$), then

$$\xi = O(n^2 \log_2^2 n),\ \sigma = O(n^2 \log_2(\log_2 n)).$$

Proof. Among the total $\mathbf{l}_{\max}$ levels of the elimination tree, the $\mathbf{l}_{\max} - \mathbf{l}_s$ bottom levels are standard factorizations, and the upper $\mathbf{l}_s$ levels are structured. Each separator i at level $\mathbf{l}$ has length $n_i = O(n/2^{\lfloor \mathbf{l}/2 \rfloor})$.

With the rank assumptions in case (i), the structured operations cost $O(n_i^2)$ flops according to Theorem 3.1. The detailed flop counts can be obtained similar to the way in [49]. See Table 4.1.

	Traditional factorizations	Structured factorizations
$\mathbf{l}_{\max} = O(\log_2 n)$ levels	$\mathbf{l} - \mathbf{l}_s$ bottom levels	$\mathbf{l}_s$ upper levels
Each level $\mathbf{l} = 0, 1, \ldots, \mathbf{l}_{\max}$	$2^{\mathbf{l}}$ separators, each of size $O(n/2^{\lfloor \mathbf{l}/2 \rfloor})$	
Cost (each separator)	$O((n/2^{\lfloor \mathbf{l}/2 \rfloor})^3)$	$O(r(n/2^{\lfloor \mathbf{l}/2 \rfloor})^2)$
Cost (subtotal)	$\sum_{\mathbf{l}=\mathbf{l}_s+1}^{\mathbf{l}_{\max}} 2^{\mathbf{l}} O((n/2^{\lfloor \mathbf{l}/2 \rfloor})^3)$	$\sum_{\mathbf{l}=0}^{\mathbf{l}_s} 2^{\mathbf{l}} O(r(n/2^{\lfloor \mathbf{l}/2 \rfloor})^2)$

TABLE 4.1
Flop count for Algorithm 3.

Thus, we have the total cost for the algorithm

$$\begin{aligned}\xi &= \sum_{\mathbf{l}=\mathbf{l}_s+1}^{\mathbf{l}_{\max}} 2^{\mathbf{l}} O((n/2^{\lfloor \mathbf{l}/2 \rfloor})^3) + \sum_{\mathbf{l}=0}^{\mathbf{l}_s} 2^{\mathbf{l}} O((n/2^{\lfloor \mathbf{l}/2 \rfloor})^2) \\ &= O(2^l(2^{\frac{1}{2}(\mathbf{l}-\mathbf{l}_s)} - 1)) + O(2^{\mathbf{l}}(\mathbf{l}_s + 2)).\end{aligned}$$

Assume n and $\mathbf{l}_{\max}$ are sufficiently large. ξ is nearly optimized when the costs before and after the switch level $\mathbf{l}_s$ are equal. That is, $2^{\frac{1}{2}(\mathbf{l}-\mathbf{l}_s)} = O(\mathbf{l}_s)$, or

$$\mathbf{l} - \mathbf{l}_s = O(\log_2 \mathbf{l}_s). \tag{4.12}$$

Then

$$\xi(\mathbf{l}_s) = O(2^{\mathbf{l}}\mathbf{l}) = O(n^2 \log_2 n).$$

In such a situation, according to Theorem 3.1 and (4.12), the storage of the structured factor is

$$\begin{aligned}\sigma &= \sum_{\mathbf{l}=\mathbf{l}_s+1}^{\mathbf{l}_{\max}} 2^{\mathbf{l}} O((n/2^{\lfloor \mathbf{l}/2 \rfloor})^2) + \sum_{\mathbf{l}=0}^{\mathbf{l}_s} 2^{\mathbf{l}} O(n/2^{\lfloor \mathbf{l}/2 \rfloor}) \\ &= O(2^{\mathbf{l}}(\mathbf{l} - \mathbf{l}_s)) + O(2^{\frac{1}{2}(\mathbf{l}+\mathbf{l}_s)}) \\ &= O(2^{\mathbf{l}} \log_2 \mathbf{l}_s) = O(2^{\mathbf{l}} \log_2 \mathbf{l}) = O(n^2 \log_2 \log_2 n).\end{aligned}$$

The rest results can be shown similarly. ∎

For 2D regular grids, $\max_i n_i = n$, and A has order $N = n^2$. For problems such as 5-point discrete Laplacian in 2D, it is shown that r_l is bounded [9]. Thus, case (i) applies and we get a fast solver with nearly linear complexity ($N \log_2 N$) and nearly linear storage ($N \log_2 \log_2 N$). Moreover, Theorem 4.3 shows that we can relax the rank requirement to get similar counts. The optimality condition (4.12) can be used to roughly decide $\mathbf{l}_s$.

We can similarly prove the results in 3D.

THEOREM 4.4. *If in Theorem 4.3, the discretized matrix A is a 3D $n \times n \times n$ regular grid, and other conditions remain the same, then the counts in cases (i) and*

(ii) of Theorem 4.3 are changed to

$$\text{(i) } \xi = O(n^4),\ \sigma = O(n^3 \log_2 n) \quad \textit{and}$$
$$\text{(ii) } \xi = O(n^4 \log_2 n),\ \sigma = O(n^3 \log_2 n),$$

respectively.

REMARK 1. For 3D regular grids, $\max_i n_i = n^2$, and A has order $N = n^3$. [9] shows that for 7-point Laplacian on 3D regular grids, $\max_l \max_l r_l = O(n^{1/2})$, and $O(n^4)$ complexity direct solvers are then possible. On the other hand, Theorem 4.4 indicates that with a much looser rank requirement $\max_l \max_l r_l = O(n)$ as in case (ii), we can still get similar complexity.

The structured algorithm in [49] is specifically designed for 2D regular grids. As compared with that solver, Algorithm 3 has some advantages. One is that this new algorithm is much simpler because the extend-add operation only uses the regular dense matrix addition. Algorithm 3 is also more general and applies to irregular grids. Furthermore, the version `rsmf` is more robust, and the version `ssmf` has better scalability. For a symmetric matrix A, the structured factor $\mathcal{L}$ from `rsmf` is always guaranteed to exist and $\mathcal{L}\mathcal{L}^T$ is guaranteed to be positive definite. The tradeoff is that there is an extra $\log_2 N$ factor in the factorization flop count (in 2D), and an extra $O(\log_2 \log_2 N)$ memory requirement. However, since the implementation is significantly simpler, the actual running time is still competitive when taking advantage of BLAS3 dense matrix operations. The storage count is nearly linear in N in practice. For example, for N as large as 10^{300}, we still have $\log_2 \log_2 N < 10$.

5. Structured multifrontal solution. After the factorization, we have $A \approx \mathcal{L}\mathcal{L}^T$ where $\mathcal{L}$ is either from the robust structured multifrontal factorization (called *triangular structured multifrontal factor*) or from the scalable ULV-type structured multifrontal factorization (called *ULV-type structured multifrontal factor*). We solve two structured systems

$$\mathcal{L}\mathbf{y} = \mathbf{b}, \tag{5.1}$$

$$\mathcal{L}^T\mathbf{x} = \mathbf{y}. \tag{5.2}$$

For convenience, assume $\mathbf{b}, \mathbf{x}, \mathbf{y}$ are partitioned conformably according to the sizes of the separators. For example, $\mathbf{b} = \left(\mathbf{b}_1^T, \mathbf{b}_2^T, \ldots, \mathbf{b}_m^T\right)^T$ with the length of $\mathbf{b}_i$ equal to the number of mesh points in separator i. Since the situation when the elimination of a node i is nonstructured (node i is before the switching level) is trivial, when describing the algorithm we only focus on structured solutions.

5.1. Triangular structured multifrontal solution. When $\mathcal{L}$ is given by the robust structured multifrontal factorization, the diagonal blocks of $\mathcal{L}$ are in HSS forms and the lower off-diagonal blocks are in compressed forms. See (4.3)-(4.5).

The solution of (5.1) with forward substitution involves forward (or postordering) traversal of the assembly tree. For a node i of the assembly tree, assume $\{s_1, s_2, \ldots, s_\gamma\}$ is the set of upper level neighbor separators of separator i. According to (4.3), we need to solve a system of the following form for $\mathbf{y}_i$:

$$\begin{pmatrix} \mathbf{L}_{i,i} & 0 \\ \mathbf{L}_{\{s_1,s_2,\ldots,s_\gamma\},i} & I \end{pmatrix} \begin{pmatrix} \mathbf{y}_i \\ \tilde{\mathbf{b}}_s \end{pmatrix} = \begin{pmatrix} \mathbf{b}_i \\ \mathbf{b}_s \end{pmatrix}, \tag{5.3}$$

where $\mathbf{b}_s$ is related to $\mathbf{b}_{s_1}, \mathbf{b}_{s_2}, \ldots, \mathbf{b}_{s_\gamma}$ (the separators $s_1, s_2, \ldots, s_\gamma$ are connected to i and partially contribute to b_s). Here, $\mathbf{b}_i$ is either from $\mathbf{b}$ (when i is a leaf), or is an

updated vector due to the solution steps associated with lower level nodes (we still use b_i for notational convenience; see (5.4)).

We first solve $\mathbf{L}_{i,i}y_i = b_i$ with a lower triangular HSS solver similar to the one in [38]. Then $\mathbf{L}_{\{s_1,s_2,\ldots,s_\gamma\},i}y_i$ is the contribution of separator i to its neighbors. That is, we update b_s by

$$\mathbf{b}_s \leftarrow \tilde{\mathbf{b}}_s = \mathbf{b}_s - \mathbf{L}_{\{s_1,s_2,\ldots,s_\gamma\},i}\mathbf{y}_i = \mathbf{b}_s - U_{k+1}B_k^T U_k^T y_i. \tag{5.4}$$

Again, U_{k+1} and B_k are explicitly available, but not U_k. However, the HSS solution of $\mathbf{L}_{i,i}\mathbf{y}_i = \mathbf{b}_i$ partially forms $U_k^T y_i$ [38]. In another word, U_k is recursively defined by lower level U generators whose products with parts of y_i are already available. Thus, $\mathbf{b}_s$ can be convenient computed, and is then used to update $\mathbf{b}_{s_1}, \mathbf{b}_{s_2}, \ldots, \mathbf{b}_{s_\gamma}$.

In the backward substitution stage for solving (5.2), we traverse the elimination tree top-down. Similarly, for each node i, we need to solve a system of the following form for $\mathbf{x}_i$:

$$\begin{pmatrix} \hat{\mathbf{L}}_i^T & \hat{\mathbf{L}}_i^{-1}U_kB_kU_{k+1}^T \\ & I \end{pmatrix} \begin{pmatrix} \mathbf{x}_i \\ \mathbf{x}_s \end{pmatrix} = \begin{pmatrix} \mathbf{y}_i \\ \mathbf{x}_s \end{pmatrix},$$

where $\mathbf{x}_s$ is already available from solution steps associated with upper level separators. We update $\mathbf{y}_i$ with $\mathbf{x}_s$ and then solve an upper triangular HSS system $\hat{\mathbf{L}}_i^T\mathbf{x}_i = \mathbf{y}_i - \hat{\mathbf{L}}_i^{-1}U_kB_kU_{k+1}^T\mathbf{x}_s$.

Note that the space of $\mathbf{b}$ can be used to store $\mathbf{y}$ and then $\mathbf{x}$. After all the updates and solutions are performed, $\mathbf{b}$ is transformed to $\mathbf{x}$.

5.2. ULV-type structured multifrontal solution. The overall framework of the ULV-type structured multifrontal solution is similar to the previous solution type. But there are some important differences.

In the forward substitution stage, similar to (5.3), we need to solve

$$\begin{pmatrix} \hat{\mathbf{L}}_i & \\ U_{k+1}B_k^TU_k^T\hat{\mathbf{L}}_i^{-T} & I \end{pmatrix} \begin{pmatrix} \mathbf{y}_i \\ \tilde{\mathbf{b}}_s \end{pmatrix} = \begin{pmatrix} \mathbf{b}_i \\ \mathbf{b}_s \end{pmatrix}.$$

Here, $\hat{\mathbf{L}}_i\mathbf{y}_i = \mathbf{b}_i$ is solved with the generalized HSS solution procedure in section 2.3.3.

The update

$$\mathbf{b}_s \leftarrow \tilde{\mathbf{b}}_s = \mathbf{b}_s - U_{k+1}B_k^TU_k^T\hat{\mathbf{L}}_i^{-T}y_i \tag{5.5}$$

is not directly computed by computing $\hat{\mathbf{L}}_i^{-T}y_i$. Instead, we can take advantage of the ULV solution step of $\hat{\mathbf{L}}_i\mathbf{y}_i = \mathbf{b}_i$ and use the following result.

THEOREM 5.1. *Let $\mathbf{y}_{i,k}$ be the solution piece obtained by the solution procedure in section 2.3.3 (as y_k in (2.12)), where k is the root of the HSS tree of $F_{i,i} \approx \hat{\mathbf{L}}_i\hat{\mathbf{L}}_i^T$. Then*

$$U_k^T\hat{\mathbf{L}}_i^{-T}\mathbf{y}_i = \bar{U}_k^TL_k^{-T}\mathbf{y}_{i,k}. \tag{5.6}$$

Thus, (5.5) can be computed as

$$\tilde{\mathbf{b}}_s = \mathbf{b}_s - U_{k+1}B_k^T\bar{U}_k^TL_k^{-T}\mathbf{y}_{i,k}. \tag{5.7}$$

Proof. For notational convenience in this proof, we write $\hat{\mathbf{L}}_i$ as $\hat{\mathbf{L}}$, $\mathbf{y}_i$ as y, and $\mathbf{y}_{i,j}$ as y_j. Thus we keep the notation consistent with that in section 2.3.3. According

to (2.11) and (2.13),

$$
\begin{aligned}
&\left(U_k^{(l+1)}\right)^T (\hat{\mathbf{L}}^{(l+1)})^{-T} y^{(l+1)} \\
&= \left(U_k^{(l+1)}\right)^T \left(X_l^{-1} P_l^T \begin{pmatrix} I & \\ & \hat{\mathbf{L}}^{(l)} \end{pmatrix} P_l\right)^{-T} P_l^T \begin{pmatrix} \bar{y}^{(l)} \\ y^{(l)} \end{pmatrix} \\
&= \left(U_k^{(l+1)}\right)^T X_l^T P_l^T \begin{pmatrix} I & \\ & (\hat{\mathbf{L}}^{(l)})^{-T} \end{pmatrix} P_l P_l^T \begin{pmatrix} \bar{y}^{(l)} \\ y^{(l)} \end{pmatrix} \\
&= \begin{pmatrix} 0 & \left(U_k^{(l)}\right)^T \end{pmatrix} \begin{pmatrix} I & \\ & (\hat{\mathbf{L}}^{(l)})^{-T} \end{pmatrix} \begin{pmatrix} \bar{y}^{(l)} \\ y^{(l)} \end{pmatrix} \\
&= \left(U_k^{(l)}\right)^T (\hat{\mathbf{L}}^{(l)})^{-T} y^{(l)}.
\end{aligned}
$$

Thus, $(U_k^{(l_{\max})})^T (\hat{\mathbf{L}}^{(l_{\max})})^{-T} y^{(l_{\max})} = (U_k^{(0)})^T (\hat{\mathbf{L}}^{(0)})^{-T} y^{(0)}$. On the other hand, $U_k^{(l_{\max})} = U_k$, $\hat{\mathbf{L}}^{(l_{\max})} = \hat{\mathbf{L}}$, $y^{(l_{\max})} = y$, $U_k^{(0)} = \bar{U}_k$, $\hat{\mathbf{L}}^{(l)} = L_k$, and $y^{(0)} = y_k$. Therefore, (5.6) holds.

Then, (5.5) and (5.6) lead to (5.7). Since L_k is a much smaller triangular matrix, (5.7) can be quickly computed. ∎

In the backward substitution stage, we solve systems of the following form for $\mathbf{x}_i$:

$$
\begin{pmatrix} \hat{\mathbf{L}}_i^T & \hat{\mathbf{L}}_i^{-1} U_k B_k U_{k+1}^T \\ & I \end{pmatrix} \begin{pmatrix} \mathbf{x}_i \\ \mathbf{x}_s \end{pmatrix} = \begin{pmatrix} \mathbf{y}_i \\ \mathbf{x}_s \end{pmatrix},
$$

where $\mathbf{x}_s$ is already available from solution steps associated with upper level separators. First, update

$$
\mathbf{y}_i \leftarrow \tilde{\mathbf{y}}_i = \mathbf{y}_i - \hat{\mathbf{L}}_i^{-1} U_k B_k U_{k+1}^T \mathbf{x}_s.
$$

Similar to Theorem 5.1, we can prove that $\tilde{\mathbf{y}}_i$ can be computed quickly as follows.

THEOREM 5.2. *The computation of* $\tilde{\mathbf{y}}_i = \mathbf{y}_i - \hat{\mathbf{L}}_i^{-1} U_k B_k U_{k+1}^T \mathbf{x}_i$ *can be done by updating the piece*

$$
\mathbf{y}_{i,k} \leftarrow \tilde{\mathbf{y}}_{i,k} = \mathbf{y}_{i,k} - L_k^{-1} \bar{U}_k B_k U_{k+1}^T \mathbf{x}_s,
$$

where $\mathbf{y}_{i,k}$ *is given in Theorem 5.1.*

Then a ULV-type upper triangular HSS system $\hat{\mathbf{L}}_i^T \mathbf{x}_i = \tilde{\mathbf{y}}_i$ is solved.

The complexity of the solution process is also almost linear in N^2 in practice.

THEOREM 5.3. *Under the same conditions as in Theorem* **??**, *the cost to solve (5.1) or (5.2) is* $O(N^2 \log_2(r \log_2 N))$.

Solution cost:

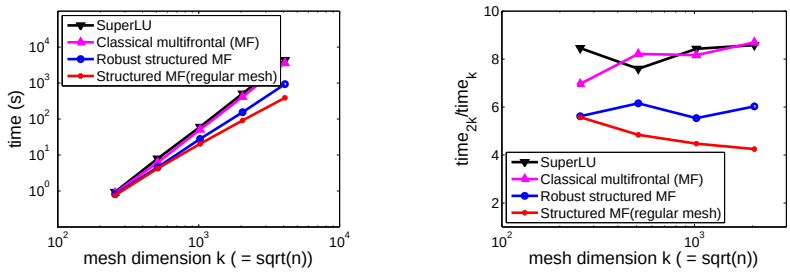
$$
\begin{aligned}
&= \sum_{i=l_s/3+1}^{l/3} 2^{3i} (2^{l/3-i})^4 + \sum_{i=l_s/3+1}^{l/3} 2^{3i+1} (2^{l/3-i-1})^4 + \sum_{i=l_s/3+1}^{l/3} 2^{3i+2} (2^{l/3-i-2})^4 \\
&+ \sum_{i=l_s/3+1}^{l/3} 2^{3i} (2^{l/3-i})^3 + \sum_{i=0}^{l_s/3} 2^{3i+1} (2^{l/3-i-1})^3 + \sum_{i=0}^{l_s/3} 2^{3i+2} (2^{l/3-i-2})^3 \\
&= 2^l (2^{\frac{1}{3}(l-l_s)} - 1) + 2^l l \\
&= O(N^3 \log_2 N)
\end{aligned}
$$

6. Numerical experiments. !!!Not finished. All the tests need to be redone!!!

6.1. Implementation issues. The new algorithm applies to general irregular meshes. In fact, it can even factorize arbitrary SPD sparse matrices, since the ordering of the rows and columns can be done on the connectivity graph of the matrix. Various ordering packages can be used. We build a nested dissection code based on the toolbox `MESHPART` [27] which uses `METIS` [42].

6.2. Poisson's Equations and Interface Problems with Large Jumps in Coefficients. We first verify the feasibility of nearly linear time sparse direct solution using a standard Poisson's equation in two dimensions. Our robust (partially) structured multifrontal test code is compared with `SuperLU` [**?**], the classical multifrontal method (MF), and our early (non-robust) structured multifrontal solver for regular meshes [49]. Note that the new robust structured code is more general and does not specifically requires the mesh to be regular. See Figure 6.1.

Similarly, we solver the 3D interface problem (**??**) with large jumps in the coefficient [**?**]. The smaller δ is, the more ill conditioned the problem is. Our robust structured solver attains satisfactory speedup with modest accuracy, and is relatively insensitive to δ. See Table 6.1. We plan to improve the algorithm using MHS structures and to solve larger problems.



Timing, with relative tolerance $\tau = 10^{-6}$ How the time scales when n doubles

FIG. 6.1. *Direct solution of Poisson's equations*

n ($\times 10^5$)	1.70	2.75	5.37	12.7
MF ($\times 10^{11}$)	0.91	4.21	12.5	41.0
New($\times 10^{11}$)	0.80	2.44	6.44	17.2

(a) Flop count ($\delta = 10^{-8}$)

n ($\times 10^5$)	1.70	2.75	5.37	12.7
MF ($\times 10^8$)	1.11	2.01	4.20	9.21
New($\times 10^8$)	1.05	1.39	2.34	4.60

(b) Storage (number of nonzeros)

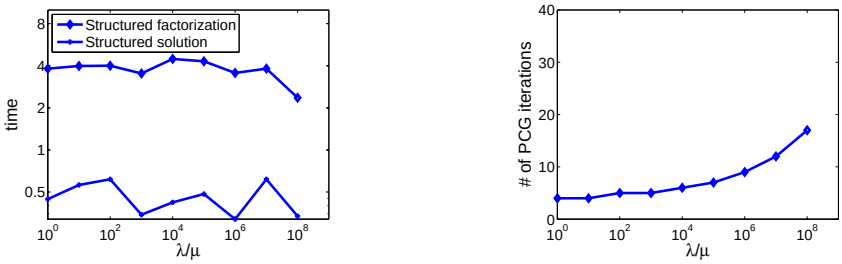
δ	10^{-2}	10^{-4}	10^{-6}	10^{-8}
Flops ($\times 10^{11}$)	9.84	8.90	8.15	6.44
Storage($\times 10^8$)	2.71	2.60	2.50	2.34

(c) New scheme (fixed n, different δ)

TABLE 6.1

*Solution of (**??**) in 3D (with jumps in coefficients) using the classical multifrontal factorization (MF) and our new robust structured factorization (New) with a relative tolerance $\tau = 10^{-3}$.*

6.3. Nearly Incompressible Linear Elasticity Equations. In structural mechanics, one often needs to solve linear elasticity equations. Standard solvers including multigrid often suffer from the deterioration of the convergence rate for large λ/μ or near the incompressible limit. For the equation (**??**), Table **??** shows the low-rank property for various λ/μ. Our robust structured factorization test code shows efficiency similar to Figure 6.1. Hence, we only demonstrate its performance for λ/μ across large scales in Figure 6.2.



Timing of the robust structured solver Number of PCG iterations to reach 10^{-15} accuracy

FIG. 6.2. *Solving and preconditioning the elasticity equation (??) with different λ/μ, where in (c), the structured preconditioners are obtained by setting the maximum numerical ranks to be a small number* 12.

6.4. Time harmonic Maxwell equations. The problems of electromagnetic fields are governed by Maxwell equations. The numerical solution of time harmonic Maxwell equations (and also Helmholtz equations, etc.) has a lot of challenges. One is that, the solution is smooth and oscillatory away from boundaries and interfaces. The approximation of oscillations generally requires sufficiently small mesh element sizes compared to the wavelengh of the solution. This leads to a large number of mesh points, especially for high wave numbers [**?**, **?**, **?**, **?**, **?**, **?**]. In addition, the resulting discretized linear systems are often severely ill conditioned. Iterative methods with many existing preconditioners including multigrid may fail to converge quickly for these problems. It is well-known that for high wave numbers, standard multigrid often encounters difficulty in approximating the problem on coarse grids, and generally, complicated implementations of multigrid are needed [**?**, **?**, **?**].

On the other hand, our preconditioners are based on structured approximate factorizations and approximate information at all grid points are used to predict the solutions. Our preliminary tests indicate that our algorithms are relatively insensitive to wave numbers, when a discretized matrix is obtained for a wave number with appropriate mesh sizes. For example, consider the following time harmonic Maxwell equations

$$\begin{aligned} \nabla \times \nabla \times \mathbf{E} - k^2 \mathbf{E} &= \mathbf{J} \text{ in } \Omega, \\ \mathbf{E} \times \mathbf{n} &= \mathbf{g} \text{ on } \partial\Omega, \end{aligned} \tag{6.1}$$

where $\mathbf{n}$ is the unit outer normal of $\partial\Omega$, and appropriate parameters are defined similar to those in [21]. *To be done.....* A preliminary `Matlab` code of our preconditioner (denoted `NEW`) is tested. The code is a black box implementation for 2D test problems but takes any sparse matrix as the input. Our code `NEW` is compared with the highly tuned `Hypre` [**?**, **?**, **?**] implementation of the multigrid type auxiliary subspace preconditioner [**?**, **?**] given in the `PHG` package (in C and Fortran) [**?**, **?**]. Even our basic `Matlab` implementation starts to outperform the `Hypre` preconditioner for not very large matrices, especially when k is relatively large (corresponding to a large wave number) with sufficiently large number of mesh points used in the discretization. `PHG` with the `Hypre` preconditioner dramatically slows down when k is large. However, the impact of a large k on our new algorithm is much less significant.

To be done.....

7. Conclusions.

k		2	4	8	16	32
Matrix dimension		84608	100904	146714	202456	272986
PHG w/ Hypre	Time (s)	45.0	126.7	1809.8	14331.2	×
	# of iterations	19	56	756	5171	×
NEW	Time (s)	10.6 [291.5]	10.8 [419.7]	24.3 [530.4]	36.6 [1364.0]	82.3 [3410.7]
	# of iterations	5	4	7	6	7

("×": failure to converge in a reasonable number of steps)

TABLE 6.2

Iterative solutions of (6.1) with preconditioners `Hypre` *and* `NEW` *to reach nearly same accuracy for each same matrix, where for* `NEW`*, the number in* [ ] *denotes the factorization time for generating the preconditioner. (This initial test intends to compare the methods on same matrices, and not to focus on good choices of matrices for different wave numbers.)*

Acknowledgments.

REFERENCES

[1] M. A. Ajiz and A. Jennings, *A robust incomplete Choleski-conjugate gradient algorithm*, Internat. J. Numer. Methods Engrg., 20 (1984), pp. 949–966.

[2] Z. Bai, J. Demmel, J. Dongarra, A. Ruhe, and H. Vorst, eds., *Templates for the solution of algebraic eigenvalue problems: a practical guide*, SIAM, Philadelphia, 2000.

[3] M. Bebendorf, *Efficient inversion of Galerkin matrices of general second-order elliptic differential operators with nonsmooth coefficients*, Math. Comp., 74 (2005), pp. 1179–1199.

[4] M. Bebendorf and W. Hackbusch, *Existence of $\mathcal{H}$-matrix approximants to the inverse FE-matrix of elliptic operators with L^∞-Coefficients*, Numer. Math., 95 (2003), pp. 1–28.

[5] M. Benzi, J. K. Cullum, and M. Tuma, *Robust approximate inverse preconditioning for the conjugate gradient method*, SIAM J. Sci. Comput. 22 (2000), pp. 1318–1332.

[6] M. Benzi and M. Tuma, *A robust incomplete factorization preconditioner for positive definite matrices*, Numer. Linear Algebra Appl., 10 (2003), pp. 385–400.

[7] S. Börm, *Construction of data-sparse $\mathcal{H}^2$-matrices by hierarchical compression*, SIAM J. Sci. Comput., 31 (2009), pp. 1820–1839.

[8] S. Börm and W. Hackbusch, *Data-sparse approximation by adaptive $\mathcal{H}^2$-matrices*, Computing, 69 (2002), pp. 1–35.

[9] S. Chandrasekaran, P. Dewilde, and M. Gu, *On the numerical rank of the off-diagonal blocks of Schur complements of discretized elliptic PDEs*, preprint, 2007.

[10] S. Chandrasekaran, P. Dewilde, M. Gu, W. Lyons, and T. Pals, *A fast solver for HSS representations via sparse matrices*, SIAM J. Matrix Anal. Appl., 29 (2006), pp. 67–81.

[11] S. Chandrasekaran, P. Dewilde, M. Gu, and T. Pals, *A fast ULV decomposition solver for hierarchically semiseparable representations*, SIAM J. Matrix Anal. Appl., 28 (2006), pp. 603–622.

[12] S. Chandrasekaran, P. Dewilde, M. Gu, T. Pals, and A.-J. van der Veen, *Fast stable solver for sequentially semi-separable linear systems of equations*, in High Performance Computing - HiPC 2002: 9th International Conference, Lecture Notes in Comput. Sci. 2552, Springer-Verlag, Heidelberg, 2002, pp. 545–554.

[13] S. Chandrasekaran, P. Dewilde, M. Gu, T. Pals, X. Sun, A.-J. van der Veen, and D. White, *Fast stable solvers for sequentially semi-separable linear systems of equations and least squares problems*, Technical report, University of California, Berkeley, CA, 2003.

[14] S. Chandrasekaran, P. Dewilde, M. Gu, T. Pals, X. Sun, A.-J. van der Veen, and D. White, *Some fast algorithms for sequentially semiseparable representations*, SIAM J. Matrix Anal. Appl, 27 (2005), pp. 341-364.

[15] S. Chandrasekaran, M. Gu, X.S. Li, and P. Vesselevski, *Schur-monotonic semi-separable approximations of symmetric positive definite matrices*, Research Notes, 2006.

[16] S. Chandrasekaran, M. Gu, X.S. Li, and J. Xia, *Some fast algorithms for hierarchically semiseparable matrices*, Technical Report, CAM 08-24, UCLA.

[17] S. Chandrasekaran, M. Gu, X. Sun, J. Xia, and J. Zhu, *A superfast algorithm for Toeplitz systems of linear equations*, SIAM J. Matrix Anal. Appl., 29 (2007), pp. 1247–1266.

[18] S. Chandrasekaran, M. Gu, and W. Lyons, *A fast and stable adaptive solver for hierarchically semi-separable representations*, Technical Report, UCSB Math 2004-20, U.C. Santa Barbara, 2004.

[19] S. Chandrasekaran, M. Gu, and T. Pals, *Fast and stable algorithms for hierarchically semi-*

separable representations, Technical Report, UCSB, April 2004.

[20] W. Chai and D. Jiao, *An $\mathcal{H}^2$-matrix-based integral-equation solver of reduced complexity and controlled accuracy for solving electrodynamic problems*, 57 (2009), IEEE Trans. Antennas Propagat., pp. 3147–3159.

[21] Z. Chen, L. Wang, and W. Zheng, *An adaptive multilevel method for time-harmonic Maxwell equations with singularities*, SIAM J. Sci. Comput., 29 (2007), pp. 118–138.

[22] J. Demmel, *The condition number of equivalence transformations that block diagonalize matrix pencils*, SIAM J. Numer. Anal., 20 (1983), pp. 599–610.

[23] J. Demmel, *Applied Numerical Linear Algebra*, SIAM, Philadelphia, PA, 1997.

[24] I. S. Duff and J. K. Reid, *The multifrontal solution of indefinite sparse symmetric linear equations*, ACM Bans. Math. Software, 9 (1983), pp. 302–325.

[25] Y. Eidelman and I. Gohberg, *On a new class of structured matrices*, Integral Equations Operator Theory, 34 (1999) pp. 293–324.

[26] J. A. George, *Nested dissection of a regular finite element mesh*, SIAM J. Numer. Anal., 10 (1973), pp. 345–363.

[27] J. R. Gilbert and S.-H. Teng, *MESHPART, A Matlab Mesh Partitioning and Graph Separator Toolbox*, `http://aton.cerfacs.fr/algor/Softs/MESHPART/`.

[28] L. Grasedyck, R. Kriemann, and S. Le Borne, *Parallel black box domain decomposition based $\mathcal{H}$-LU preconditioning*, Technical Report 115, Max Planck Institute for Mathematics in the Sciences, Leipzig, 2005.

[29] L. Grasedyck, R. Kriemann, and S. Le Borne, *Domain-decomposition based $\mathcal{H}$-LU preconditioners*, in Domain Decomposition Methods in Science and Engineering XVI, O.B.Widlund and D.E.Keyes (eds.), Springer LNCSE, 55 (2006), pp. 661–668.

[30] W. Hackbusch, *A Sparse matrix arithmetic based on $\mathcal{H}$-matrices. Part I: introduction to $\mathcal{H}$-matrices*, Computing, 62 (1999), pp. 89–108.

[31] W. Hackbusch and B. N. Khoromskij, *A sparse $\mathcal{H}$-matrix arithmetic. Part-II: Application to multi-dimensional problems*, Computing, 64 (2000), pp. 21–47.

[32] F. Harary, *A graph theoretic approach to matrix inversion by partitioning*, Numer. Math., 4 (1962), pp. 128–135.

[33] I. Hladik, M. B. Reed, and G. Swoboda, *Robust preconditioners for linear elasticity FEM analysis*, Internat. J. Numer. Methods Engng., 40 (1997), pp. 2109-2127.

[34] A. J. Hoffman, M. S. Martin, and D. J. Rose, *Complexity bounds for regular finite difference and finite element grids*, SIAM J. Numer. Anal., 10 (1973), pp. 364–369.

[35] I. E. Kaporin, *High quality preconditioning of a general symmetric positive definite matrix based on its $U^TU + U^TR + R^TU$-decomposition*, Numer. Linear Algebra Appl., 5 (1998) pp. 483–509.

[36] D. S. Kershaw, *The incomplete Cholesky conjugate gradient method for the iterative solution of systems of linear equations*, J. Comp. Phys., 26 (1977), pp. 43–65.

[37] J. W. H. Liu, *The multifrontal method for sparse matrix solution: Theory and practice*, SIAM Review, 34 (1992), pp. 82–109.

[38] W. Lyons, *Fast Algorithms with Applications to PDEs*, PhD thesis, University of California, Santa Barbara, June. 2005.

[39] P. G. Martinsson, V. Rokhlin, and M. Tygert, *A fast algorithm for the inversion of general Toeplitz matrices*, Comput. Math. Appl. 50 (2005), pp. 741–752.

[40] T. Manteuffel, *An incomplete factorization technique for positive definite linear systems*, Math. Comput. 34 (1980), pp. 473–497.

[41] J. A. Meijerink and H. A. van der Vorst, *An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix*, Math. Comput. 31 (1977), pp. 148–162.

[42] METIS, *Family of Multilevel Partitioning Algorithms*, `http://glaros.dtc.umn.edu/gkhome/views/metis`.

[43] S. V. Parter, *The use of linear graphs in gaussian elimination*, SIAM Rev., 3 (1961), pp. 119–130.

[44] P.-O. Persson and G. Strang, *A simple mesh generator in MATLAB*, SIAM Review, 46 (2004), pp. 329–345.

[45] M. Rezghi and S. M. Hosseini, *An ILU preconditioner for nonsymmetric positive definite matrices by using the conjugate Gram-Schmidt process*, J. Comput. Appl. Math., 188 (2006), pp. 150–164.

[46] V. Strassen, *Gaussian elimination is not optimal*, Numer. Math., 13 (1969), pp. 354–356.

[47] R. P. Tewarson, *On the product form of inverses of sparse matrices and graph theory*, SIAM Rev., 9 (1967), pp. 91–99.

[48] R. Vandebril, M. Van Barel, G. Golub, and N. Mastronardi, *A bibliography on semisep-*

arable matrices, Calcolo, 42 (2005), pp. 249–270.

[49] S. Chandrasekaran, M. Gu, and X. S. Li, J. Xia, *Superfast multifrontal method for structured linear systems of equations*, submitted to SIAM J. Matrix Anal. Appl., 2009, `http://www.math.purdue.edu/~xiaj/work/supermf.pdf`.

[50] M. Gu and J. Xia, *Robust structured factorization and preconditioning for SPD matrices*, submitted to SIAM J. Matrix Anal. Appl., 2009.